Option 1: No Errors on Data

Code:

```
# Define a function (four-variable linear in our case) to fit the data
with.
def fitting_func(B, x):
    return B[0]*x[0] + B[1]*x[1] + B[2] # A simple linear fit

# Create a model for fitting.
linear_model = Model(fitting_func)

# Create a RealData object using our initiated data from above.
data = RealData((logLData, logPData), y=logMdotData) # Data: x=logL, y=logR

# Set up ODR with the model and data.
odr = ODR(data, linear_model,
beta0=[model._results.params[1],model._results.params[2],model._results.par
ams[0]]) # Use the linear regression as an initial guess

# Run the regression.
out = odr.run()

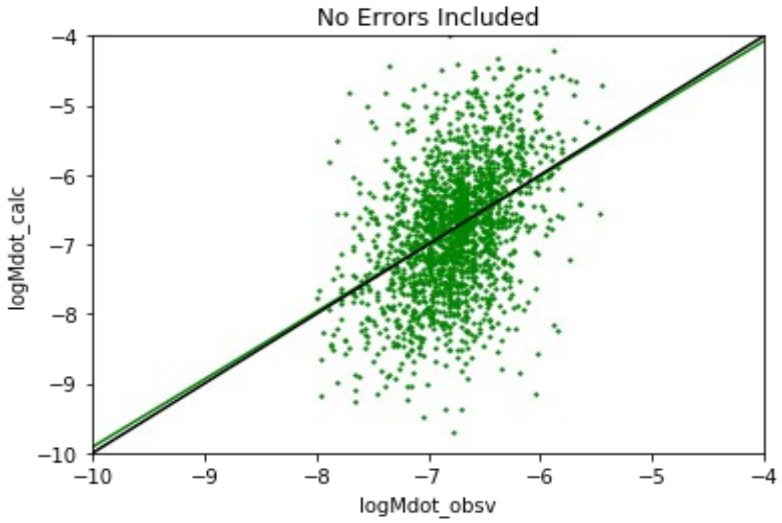
# Use the in-built pprint method to give us results.
out.pprint()
```

Out:

```
Beta: [-7.5573466  10.6568775  -3.50470881]
Beta Std Error: [0.39978479 0.52074141 0.45843275]
Beta Covariance: [[ 40.37751384 -50.33664749 -31.94687263]
 [-50.33664749  68.5064204   27.19509257]
 [-31.94687263  27.19509257  53.09312757]]
Residual Variance: 0.003958338774077242
Inverse Condition #: 0.005759344727318942
Reason(s) for Halting:
  Sum of squares convergence
```

Fit: logMdot = -7.557*logL + 10.657*logP - 3.505

Comparison to Observed: logMdot_calc = (-0.1804253209781438) + (0.9733570178831799)*logMdot_obsv



Option 1: Standard Errors on Data

```
# Define a function (four-variable linear in our case) to fit the data
with.
def fitting_func(B, x):
    return B[0]*x[0] + B[1]*x[1] + B[2] # A simple linear fit

# Create a model for fitting.
linear_model = Model(fitting_func)

# Create a RealData object using our initiated data from above.
data = RealData((logLData, logPData), y=logMdotData,
sx=(np.std(logLData),np.std(logPData)), sy =np.std(logMdotData)) # Data:
x=logL, y=logR

# Set up ODR with the model and data.
odr = ODR(data, linear_model,
beta0=[model._results.params[1],model._results.params[2],model._results.par
ams[0]]) # Use the linear regression as an initial guess

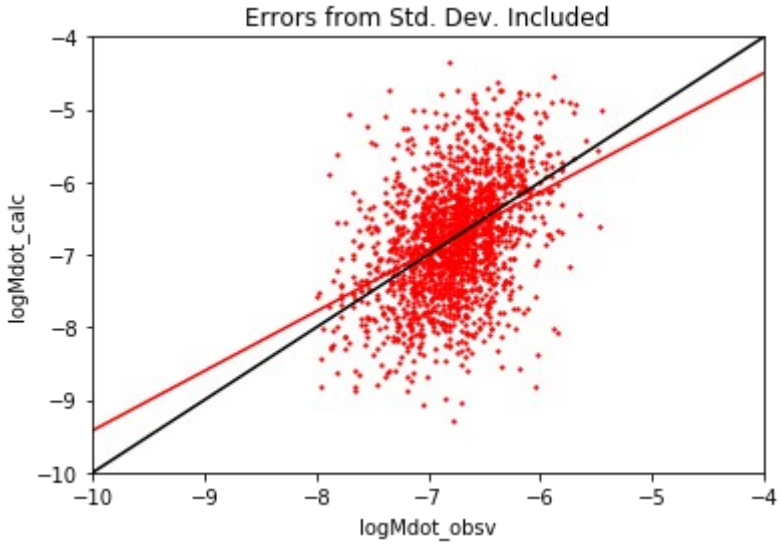
# Run the regression.
out = odr.run()

# Use the in-built pprint method to give us results.
out.pprint()
```

Out:

```
Beta: [-6.68527827  9.10645057 -3.17775207]
Beta Std Error: [0.31139733 0.38780975 0.40646441]
Beta Covariance: [[ 0.53977164 -0.6349113  -0.51047603]
 [-0.6349113   0.83717785  0.40210141]
 [-0.51047603  0.40210141  0.91965582]]
Residual Variance: 0.1796468966810999
Inverse Condition #: 0.006529552621948931
Reason(s) for Halting:
  Sum of squares convergence
```

Fit: logMdot = -6.685*logL + 9.1065*logP - 3.1777
Comparison to Observed: logMdot_calc = (-1.2150075713119945) + (0.8205826835749306)*logMdot_obsv



Option 3: “Real” Errors Used

```
# Define a function (four-variable linear in our case) to fit the data with.
def fitting_func(B, x):
    return B[0]*x[0] + B[1]*x[1] + B[2] # A simple linear fit

# Create a model for fitting.
linear_model = Model(fitting_func)
#
meanMdotError = np.mean(10**logMdotDataError/10**logMdotData)
meanLError = np.mean(10**logLDataError/10**logLData)

# Create a RealData object using our initiated data from above.
data = RealData((logLData, logPData), y=logMdotData,
sx=(meanLError,np.std(logPData)), sy =meanMdotError) # Data: x=logL, y=logR

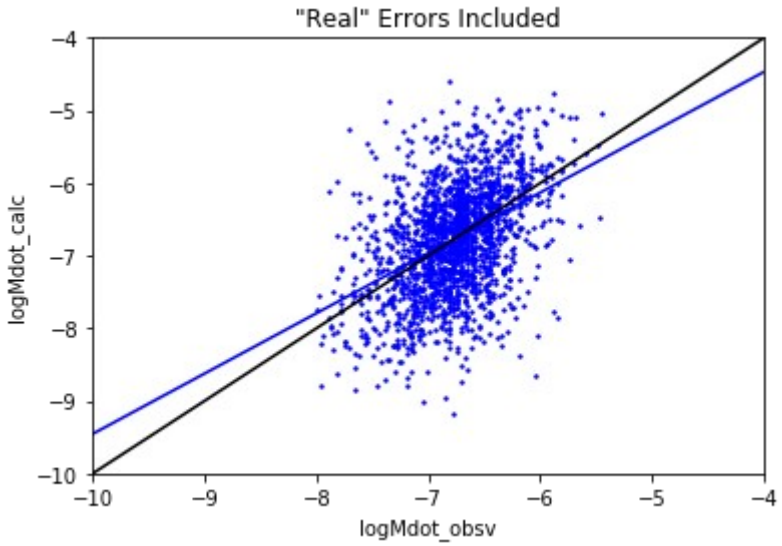
# Set up ODR with the model and data.
odr = ODR(data, linear_model,
beta0=[model._results.params[1],model._results.params[2],model._results.params[0]])
# Use the linear regression as an initial guess

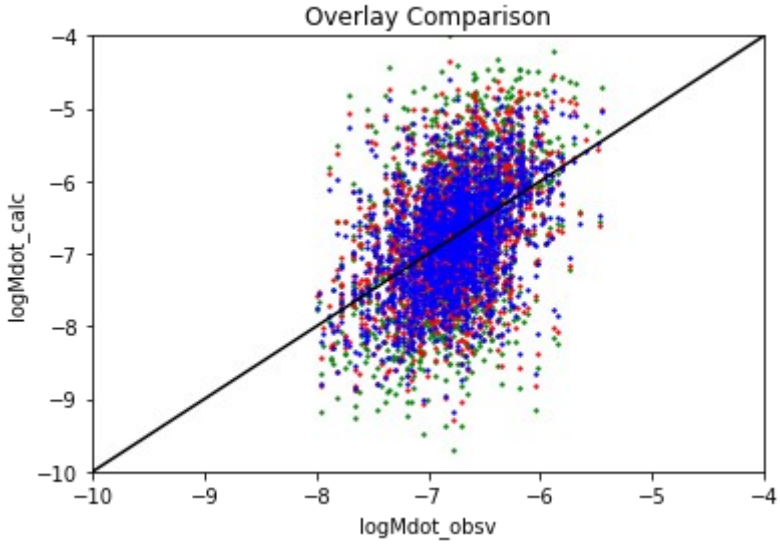
# Run the regression.
out = odr.run()

# Use the in-built pprint method to give us results.
out.pprint()

Out:
Beta: [-5.59704259  8.76534189 -6.26805678]
Beta Std Error: [0.24930287 0.34757531 0.32840067]
Beta Covariance: [[ 0.23253291 -0.30147739 -0.1585388 ]
 [-0.30147739  0.45198883  0.07136375]
 [-0.1585388   0.07136375  0.4034948 ]]
Residual Variance: 0.2672822608069985
Inverse Condition #: 0.00784548976090025
Reason(s) for Halting:
  Sum of squares convergence
```

Fit: logMdot = -5.597*logL + 8.765*logP - 6.268
Comparison to Observed: logMdot_calc = (-1.1446281248835657) + (0.8309754523304281)*logMdot_obsv




Overlay Comparison
logMdot_calc
logMdot_obsv
−4
−5
−6
−7
−8
−9
−10
−10
−9
−8
−7
−6
−5
−4